

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science
Semester	V
Subject Code & Name	CS3807 – Deep Learning Laboratory
Academic Year	2026–27
Student Name	A.Priyankaa
Roll Number	24011101001

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias

- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms

- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code

The source code and implementation are available at the following repository:

GitHub Repository

7. Experimental Results

The Single Layer Perceptron was trained on the Banknote Authentication dataset using an 80-20% split. It was trained for 20 epochs with a learning rate of 0.01. The model is trained such that the weights and bias are updated iteratively using the Perceptron learning rule till the classifier converges.

Table 1: Performance Metrics

Metric	Value
Accuracy	98.55%
Precision	0.9683
Recall	1.0000
F1-Score	0.9839

The confusion matrix obtained on the test dataset :

$$\begin{bmatrix} 149 & 4 \\ 0 & 122 \end{bmatrix}$$

Confusion Matrix

The model achieved an **accuracy of 98.55%**, correctly identifying most of the banknotes. Only four authentic notes were incorrectly classified as forged, while no forged notes were misclassified as authentic. So this gives a perfect recall of **1.0000**, which shows that no forged notes are misclassified as authentic banknotes.

8. Generated Plots with Interpretation

8.1 Feature Histograms

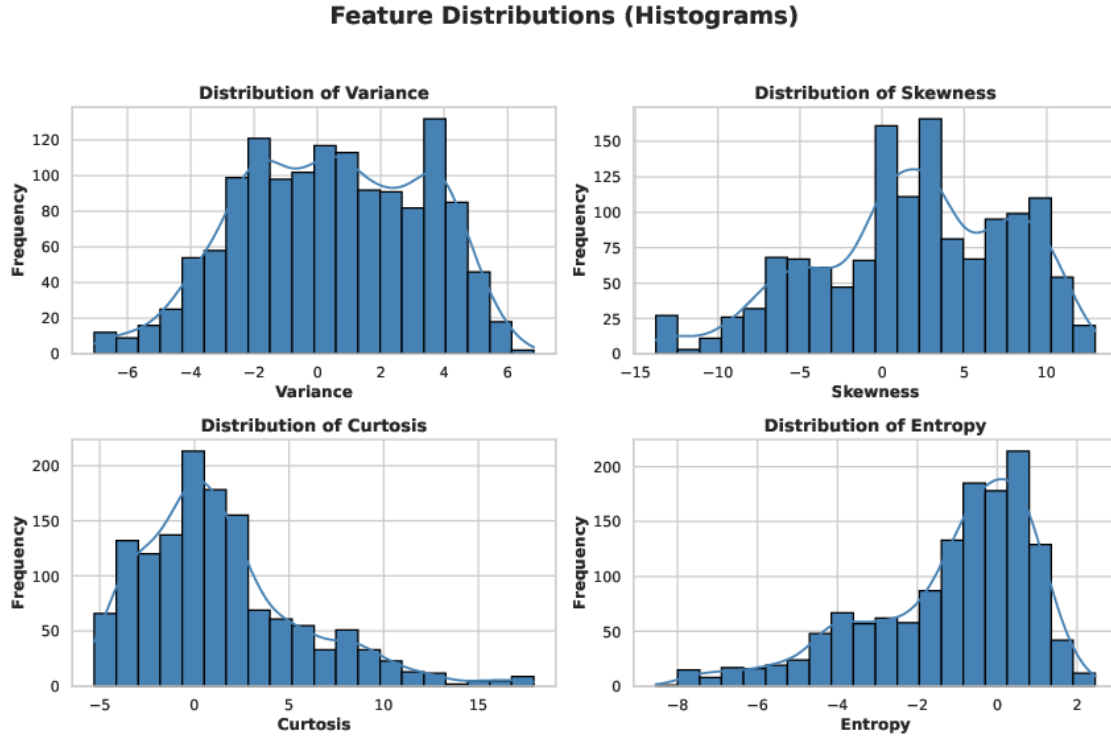


Figure 1: Feature distributions of the Banknote Authentication dataset.

The plots show the varying distributions of the four features in the dataset. Variance and skewness have a wider spread, while kurtosis and entropy have narrow spread. This indicates variance and skewness contribute more in classification and entropy contributes the least.

8.2 Correlation Heatmap

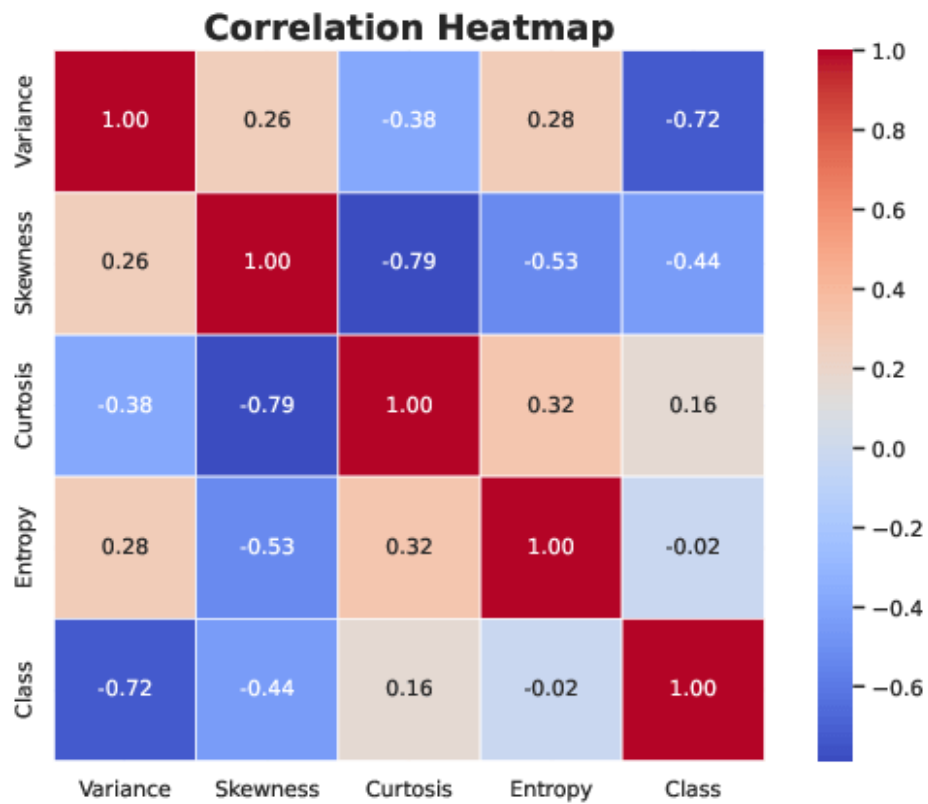


Figure 2: Correlation between the input features.

The heatmap shows the relationships among the four features. There is a strong negative correlation between skewness and kurtosis indicating both features are not required to contribute independently. The other relations are weak.

8.3 Scatter Plots

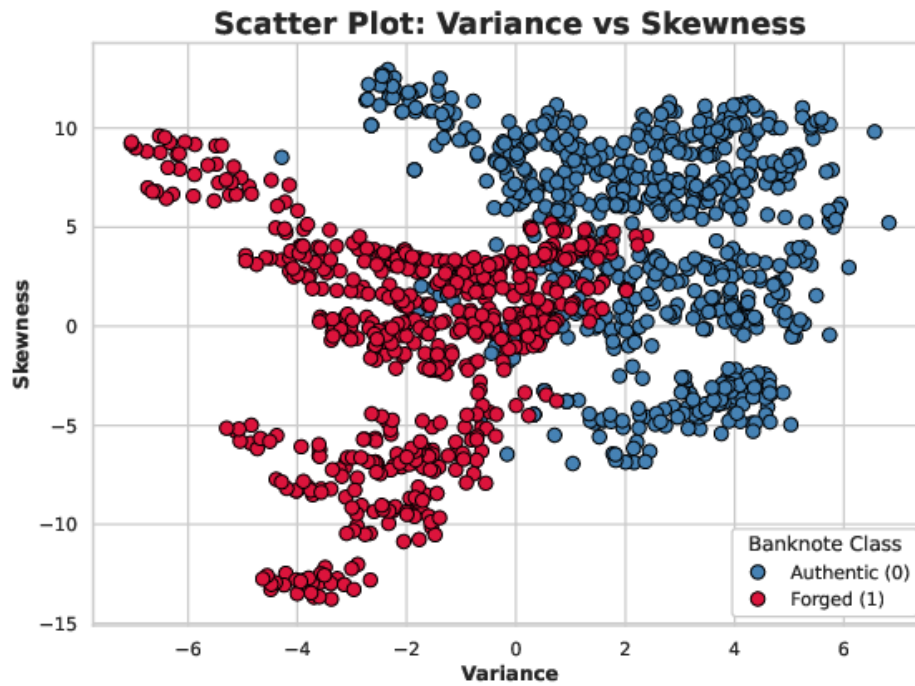


Figure 3: Scatter plot of Variance versus Skewness.

The scatter plot between variance and skewness shows that the two classes are mostly separated with a small overlapping region. This indicates that these two features are effective in distinguishing between authentic and forged banknotes.

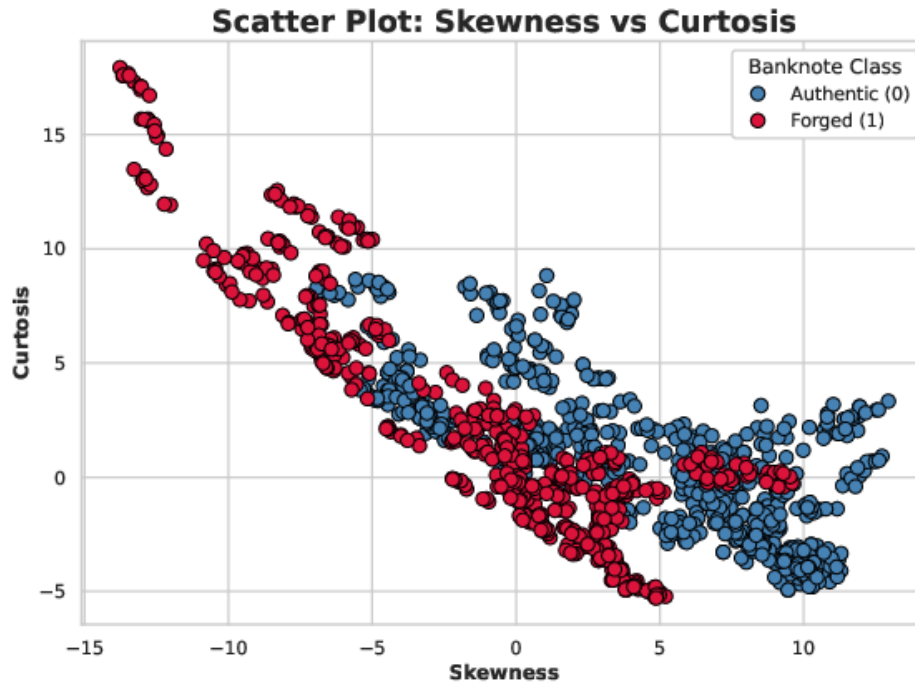


Figure 4: Scatter plot of Skewness versus Kurtosis.

The scatter plot between skewness and kurtosis shows an overlapping relationship between the two features. So they provide very less useful information for class separation.

8.4 Boxplots

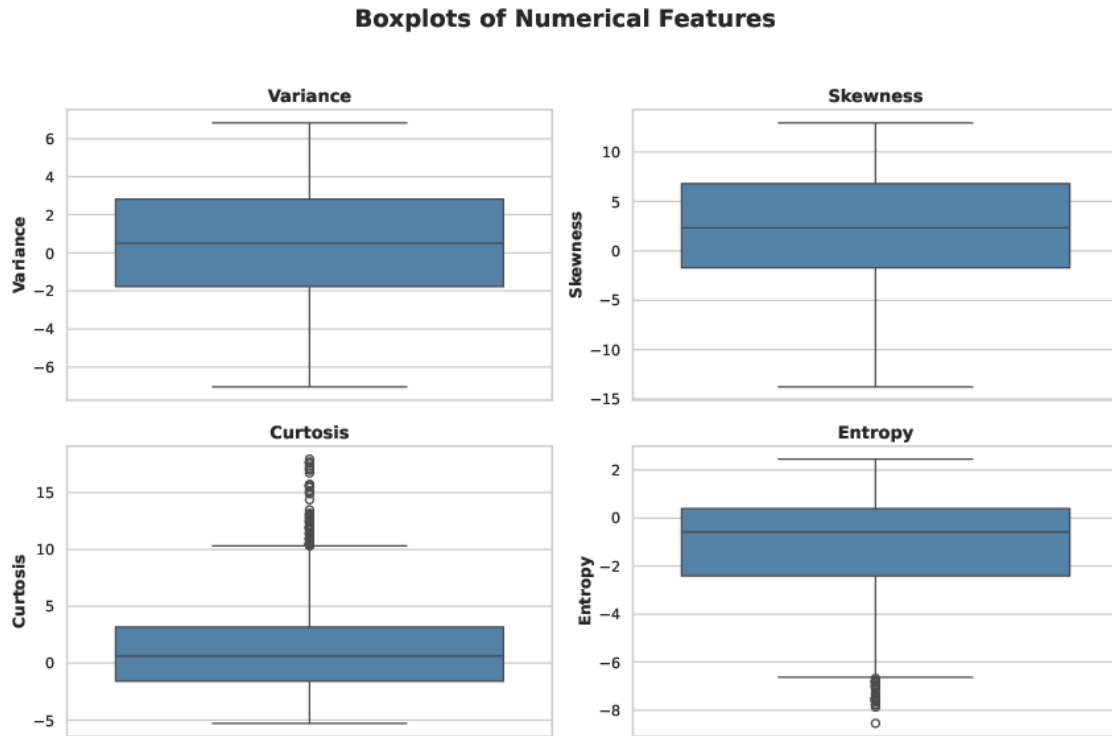


Figure 5: Boxplots of the four input features.

The boxplots compares the distribution of each feature. Variance and Skewness have a wider spread while Kurtosis and Entropy contains some outliers indicating that a few banknotes have more extreme values.

8.5 Confusion Matrix

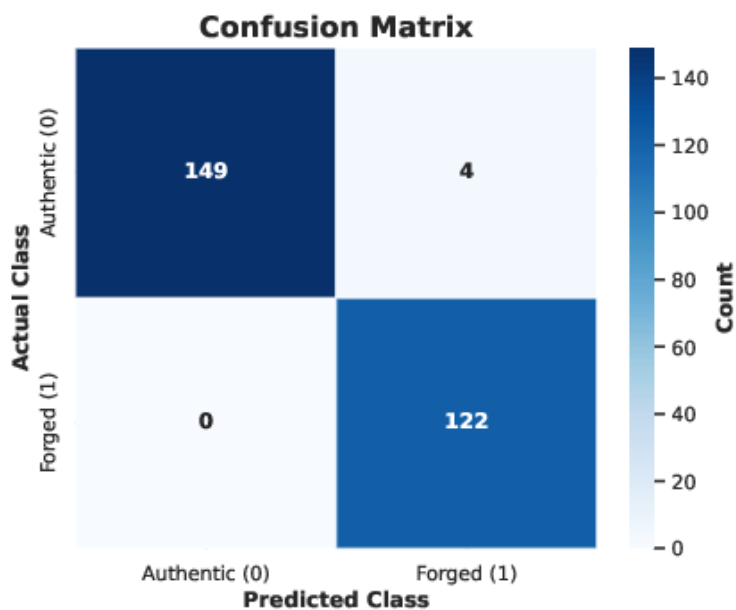


Figure 6: Confusion matrix of the trained perceptron.

The confusion matrix shows that the perceptron classified 149 authentic and all 122 forged banknotes correctly. Only 4 authentic banknotes were incorrectly classified as forged, while no forged banknotes were misclassified as authentic.

8.6 Training Error vs Epoch



Figure 7: Training error across epochs.

The training error decreases sharply during the first few epochs which shows that the perceptron quickly learns the patterns in the dataset. After the initial phase the error fluctuates within a small range indicating that the model has become stable which suggests that the dataset is mostly linearly separable.

8.7 Weight Evolution

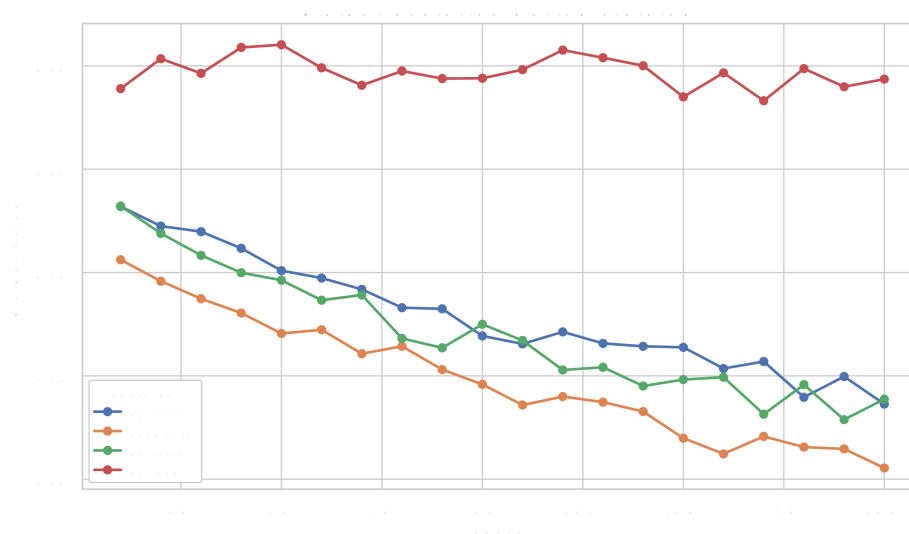


Figure 8: Evolution of feature weights during training.

The plot shows that the weights change significantly during the early epochs as the perceptron learns the features. In later epochs the values become more stable with small variations. This indicates that the model has learnt to differentiate authentic and forged notes.

8.8 Bias Evolution

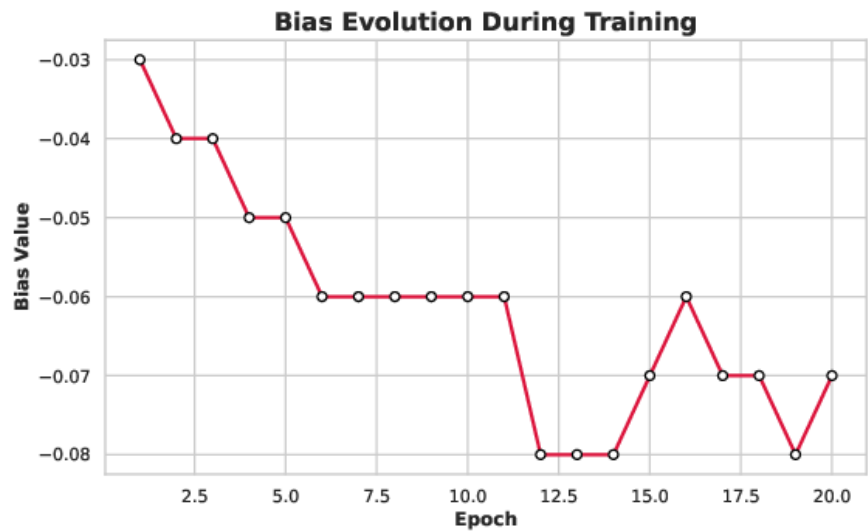


Figure 9: Bias evolution during training.

The bias value is updated to improve the position of the decision boundary. Changes occur during the initial epochs but after that the bias stabilizes and it is converged.

8.9 Learning Rate Comparison

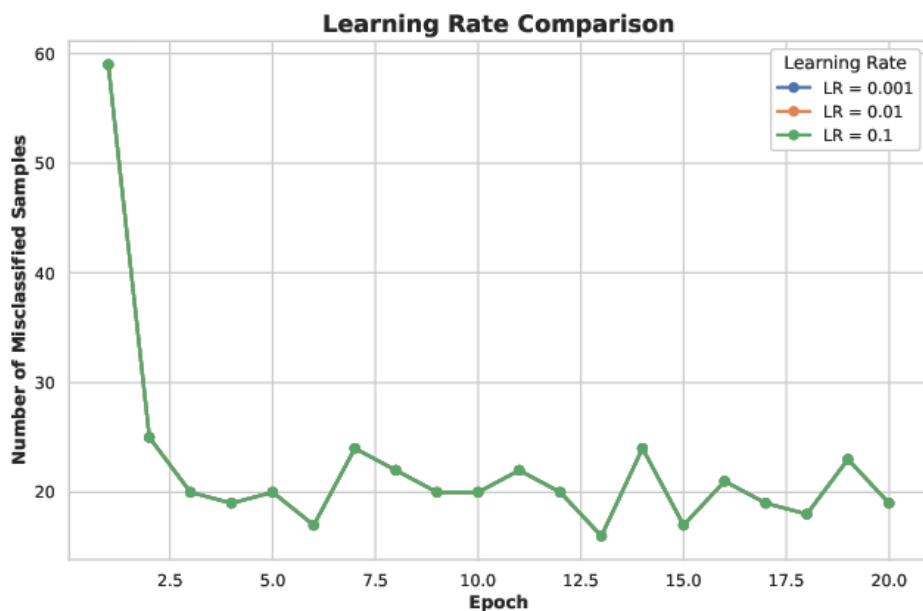


Figure 10: Training error for different learning rates.

The plot shows that all the three learning rates follows the same pattern throughout the epochs. The number of misclassified samples decreases as the perceptron converges.

9. Performance Tables

Training Summary

Dataset Size	1372 Samples
Train/Test Split	80% Training / 20% Testing
Learning Rate	0.01
Epochs	20
Final Weights	[-0.1636, -0.1946, -0.1613, -0.0064]
Final Bias	-0.07
Accuracy	98.55%
Precision	96.83%
Recall	100.00%
F1-score	98.39%

Epoch-wise Learning

Epoch	Errors	w_1	w_2	w_3	w_4	Bias
1	59	-0.0680	-0.0938	-0.0678	-0.0110	-0.03
10	20	-0.1307	-0.1541	-0.1250	-0.0060	-0.06
20	19	-0.1636	-0.1946	-0.1613	-0.0064	-0.07

10. Discussion

1. **Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.** The Step Activation Function produces a binary output which works for simple binary classification problems. But it is not differentiable and has a zero gradient which prevents backpropagation so it is not used in deep learning models.

The Sigmoid Activation Function produces continuous output values between 0 and 1 and is also differentiable. This allows neural networks to update weights efficiently during training. So Sigmoid is used in modern deep learning models.

Property	Step Function	Sigmoid Function
Output Range	$\{0, 1\}$	$(0, 1)$
Differentiable	No	Yes
Used in Backpropagation	No	Yes
Modern Deep Learning	No	Yes

2. **Compare the implementation with Scikit-learn's Perceptron.**

Model	Accuracy	Precision	Recall	F1-Score
Manual Perceptron	0.9855	0.9683	1.0000	0.9839
Scikit-learn Perceptron	0.9964	0.9919	1.0000	0.9959

The Scikit learn model achieved slightly better performance than the manually implemented one. Because it uses optimized algorithms, better convergence strategies while following the same perceptron learning rule.

3. **Repeat the experiment using learning rates 0.001, 0.01 and 0.1.**

Learning Rate	Accuracy	Precision	Recall	F1-Score
0.001	0.9855	0.9683	1.0000	0.9839
0.010	0.9855	0.9683	1.0000	0.9839
0.100	0.9855	0.9683	1.0000	0.9839

All three learning rates produced identical performance on the test dataset. This shows that the dataset is easily separable for the perceptron.

4. **Explain why a Single Layer Perceptron cannot solve the XOR problem.**

XOR problem is not linearly separable so a straight line cannot separate its two classes. Single Layer Perceptron can learn only linear decision boundaries so it cannot classify all XOR input combinations. Solving the XOR problem requires hidden layers with non linear activation functions used in multilayer Perceptrons.

5. **Study the effect of feature normalization on model convergence.**

Feature normalization scales all features to a similar range so that larger values don't dominate the learning process. Normalization resulted in faster stable convergence and without it convergence may be slower and training can become less stable.

11. Additional Tasks

1. **Code the Perceptron Learning Algorithm for OR, NOT, and AND gates using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python built-in packages.**

The Perceptron Learning Algorithm was implemented for the AND, OR and NOT gates, showing the weight updates and the decision boundary after each iteration.

1.1 AND Gate

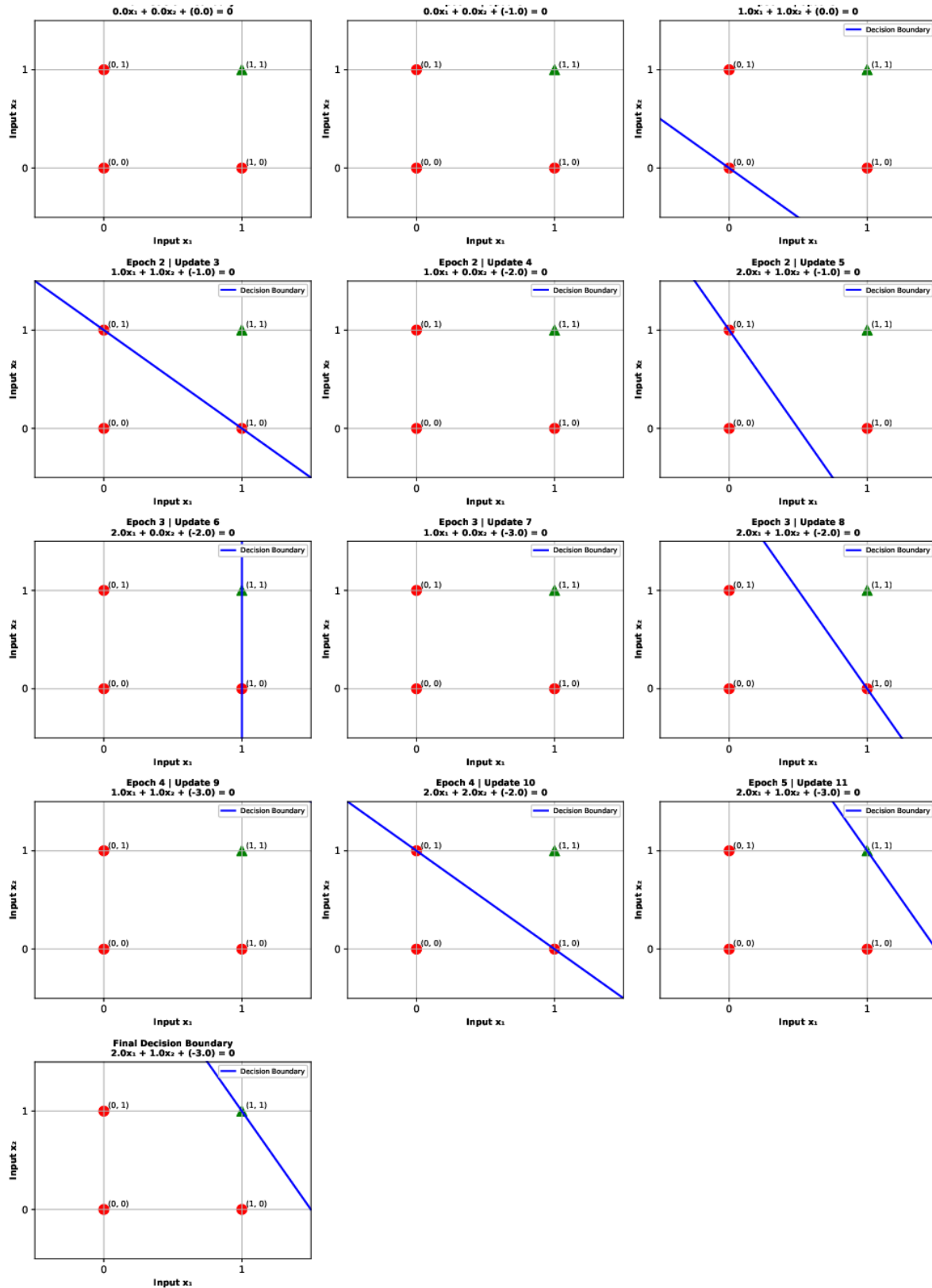


Figure 11: Decision boundary progression for the AND gate.

Table 2: Weight Update History for the AND Gate

Update	Input	Target	Prediction	Weights	Bias
1	[0,0]	0	1	[0,0]	-1
2	[1,1]	1	0	[1,1]	0
3	[0,0]	0	1	[1,1]	-1
4	[0,1]	0	1	[1,0]	-2
5	[1,1]	1	0	[2,1]	-1
6	[0,1]	0	1	[2,0]	-2
7	[1,0]	0	1	[1,0]	-3
8	[1,1]	1	0	[2,1]	-2
9	[1,0]	0	1	[1,1]	-3
10	[1,1]	1	0	[2,2]	-2
11	[0,1]	0	1	[2,1]	-3

The perceptron learned the AND gate in six epochs and correctly classified all four input combinations. This confirms that the AND gate is linearly separable.

1.2 OR Gate

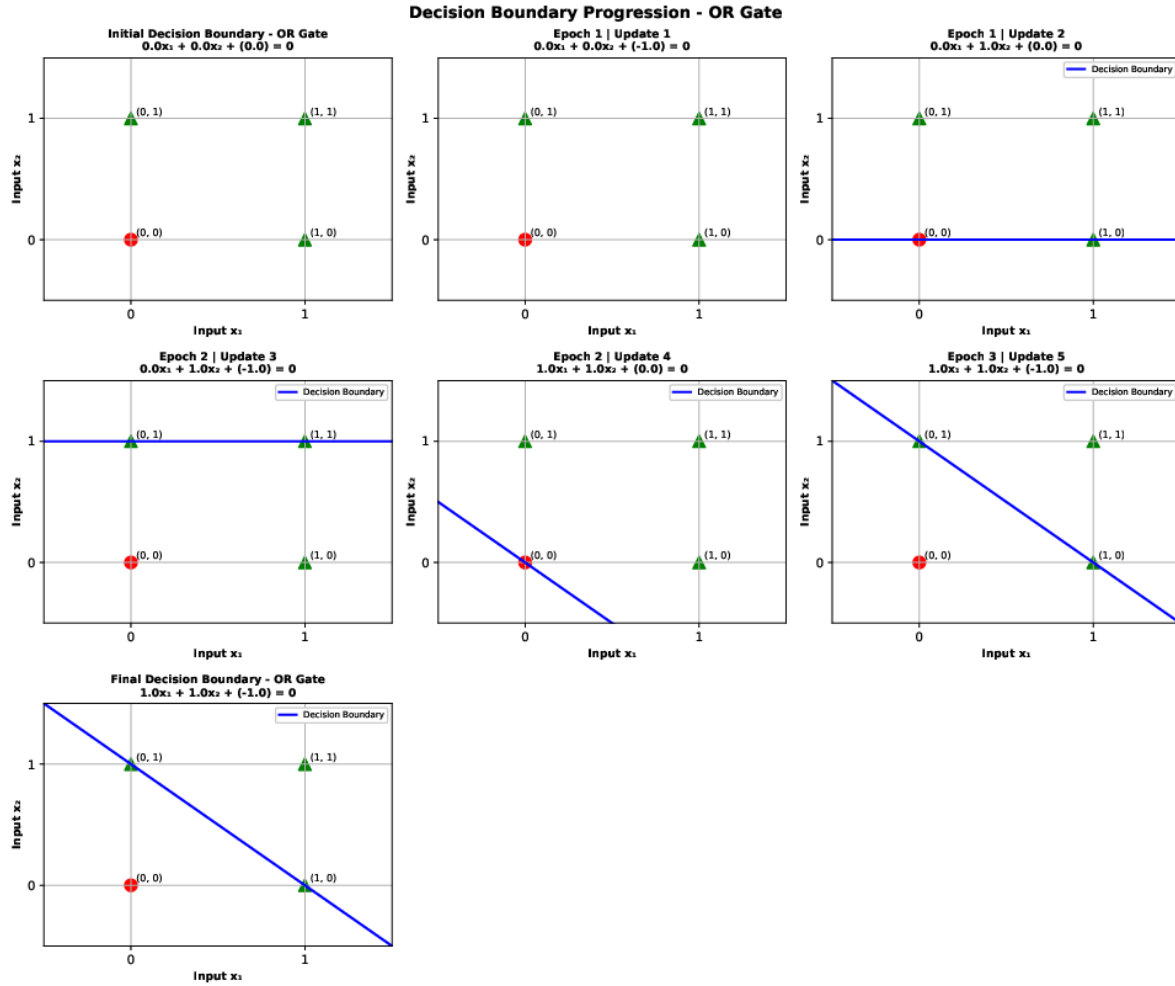


Figure 12: Decision boundary progression for the OR gate.

Table 3: Weight Update History for the OR Gate

Update	Input	Target	Prediction	Weights	Bias
1	[0,0]	0	1	[0,0]	-1
2	[0,1]	1	0	[0,1]	0
3	[0,0]	0	1	[0,1]	-1
4	[1,0]	1	0	[1,1]	0
5	[0,0]	0	1	[1,1]	-1

The perceptron learned the OR gate in four epochs and correctly classified all four input combinations. This confirms that the OR gate is linearly separable.

1.3 NOT Gate

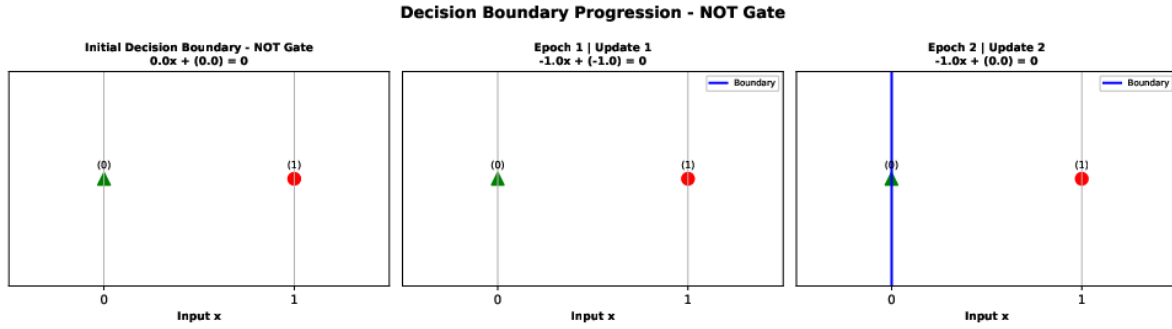


Figure 13: Decision boundary progression for the NOT gate.

Table 4: Weight Update History for the NOT Gate

Update	Input	Target	Prediction	Weight	Bias
1	[1]	0	1	[-1]	-1
2	[0]	1	0	[-1]	0

The perceptron learned the NOT gate in three epochs and correctly classified both input combinations. This shows that the NOT gate is also linearly separable.

12. Conclusion

This experiment showed how a Single Layer Perceptron can be implemented from scratch to perform binary classification. The model achieved an accuracy of 98.55% showing that the Banknote Authentication dataset is mostly linearly separable. The evaluation metrics and confusion matrix indicated that only a few samples were misclassified. Overall the model performed well.

13. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.